writing-challenges

Writing Challenges

This guide walks through creating new challenges using the `pkg/userflow` templates. It covers the full workflow from choosing a template to registering the challenge with the framework.

Step 1: Choose a Template

Select the template that matches your testing goal:

Goal	Template	Adapter Needed
Check an API health endpoint	`NewAPIHealthChallenge`	`APIAdapter`
Execute a multi-step API flow	`NewAPIFlowChallenge`	`APIAdapter`
Automate a browser workflow	`NewBrowserFlowChallenge`	`BrowserAdapter`
Verify a project builds	`NewBuildChallenge`	`BuildAdapter`
Run test suites	`NewUnitTestChallenge`	`BuildAdapter`
Run linters	`NewLintChallenge`	`BuildAdapter`
Install and launch a mobile app	`NewMobileLaunchChallenge`	`MobileAdapter`
Automate mobile interactions	`NewMobileFlowChallenge`	`MobileAdapter`
Run on-device tests	`NewInstrumentedTestChallenge`	`MobileAdapter`
	`NewDesktopLaunchChallenge`	`DesktopAdapter`

Goal	Template	Adapter Needed
Launch and verify a desktop app		
Automate desktop Web-View	`NewDesktopFlowChallenge`	`DesktopAdapter`
Test desktop IPC commands	`NewDesktopIPCChallenge`	`DesktopAdapter`
Set up test infrastructure	`NewEnvironmentSetupChallenge`	none
Tear down test infrastructure	`NewEnvironmentTeardownChallenge`	none

Step 2: Create the Adapter

Instantiate the adapter for your platform:

```
// API
api := userflow.NewHTTPAPIAdapter("http://localhost:8080")

// Browser
browser := userflow.NewPlaywrightCLIAdapter("ws://localhost:9222")

// Mobile
mobile := userflow.NewADBCLIAdapter(userflow.MobileConfig{
    PackageName:  "com.example.app",
    ActivityName: ".MainActivity",
})

// Desktop
desktop := userflow.NewTauriCLIAdapter("/path/to/binary")

// Build (choose one per toolchain)
goBuild := userflow.NewGoCLIAdapter("/path/to/project")
npmBuild := userflow.NewNPMCLIAdapter("/path/to/project")
gradleBuild := userflow.NewGradleCLIAdapter("/path/to/project",
        false)
cargoBuild := userflow.NewCargoCLIAdapter("/path/to/project")
```

Step 3: Define the Flow

For flow-based templates, create the flow definition. Flows are JSON-serializable, so they can also be loaded from configuration files.

API Flow Example

```
flow := userflow.APIFlow{
    Name: "user-management",
    Credentials: userflow.Credentials{
        Username: "admin",
        Password: "secret",
    },
    Steps: []userflow.APIStep{
        {
            Name:           "create-user",
            Method:         "POST",
            Path:           "/api/v1/users",
            Body:
        `{"name":"testuser","email":"test@example.com"}`,
            ExpectedStatus: 201,
            ExtractTo:      map[string]string{"id": "user_id"},
        },
        {
            Name:           "verify-user",
            Method:         "GET",
            Path:           "/api/v1/users/{{user_id}}",
            ExpectedStatus: 200,
            Assertions: []userflow.StepAssertion{
                {
                    Type:    "response_contains",
                    Target:  "body",
                    Value:   "testuser",
                    Message: "response should contain user name",
                },
            },
        },
    },
}
```

Browser Flow Example

```
flow := userflow.BrowserFlow{
    Name:     "signup-flow",
    StartURL: "http://localhost:3000/signup",
    Config: userflow.BrowserConfig{
        Headless:   true,
        WindowSize: [2]int{1920, 1080},
    },
```

```
    Steps: []userflow.BrowserStep{
        {Name: "fill-email", Action: "fill", Selector: "#email",
        Value: "test@example.com"},
        {Name: "fill-pass", Action: "fill", Selector:
        "#password", Value: "secret123"},
        {Name: "submit", Action: "click", Selector:
        "button[type=submit]"},
        {Name: "wait-redirect", Action: "wait", Selector:
        ".dashboard", Timeout: 5 * time.Second},
        {Name: "verify-url", Action: "assert_url", Value: "/
        dashboard"},
    },
}
```

Step 4: Create the Challenge

Combine the adapter and flow into a challenge:

```
ch := userflow.NewAPIFlowChallenge(
    "CH-USER-MGMT-001",       // unique ID
    "User Management Flow",    // display name
    "Test user CRUD lifecycle", // description
    []challenge.ID{"CH-ENV-001"}, // depends on environment setup
    api,                       // adapter
    flow,                      // flow definition
)
```

Choosing an ID

Challenge IDs must be unique within the registry. A common convention is `CH-<CATEGORY>-<NUMBER>`:

- `CH-API-001`, `CH-API-002`, ...
- `CH-BROWSER-001`, ...
- `CH-BUILD-001`, ...
- `CH-ENV-001`, `CH-ENV-TEARDOWN`

Setting Dependencies

The `deps` parameter is a slice of `challenge.ID` values that must pass before this challenge runs. Common patterns:

```
// No dependencies (root challenge)
deps := nil

// Depends on environment setup
deps := []challenge.ID{"CH-ENV-001"}

// Depends on build passing
deps := []challenge.ID{"CH-BUILD-001"}
```

```
// Multiple dependencies
deps := []challenge.ID{"CH-ENV-001", "CH-BUILD-001"}
```

Step 5: Register the Challenge

Register the challenge with the challenge registry so the runner can discover and execute it:

```
import "digital.vasic.challenges/pkg/registry"

reg := registry.New()
reg.Register(ch)
```

Or register multiple challenges:

```
reg.Register(setup)
reg.Register(buildChallenge)
reg.Register(testChallenge)
reg.Register(apiChallenge)
reg.Register(teardown)
```

The registry uses Kahn's algorithm for topological sorting, so challenges are executed in dependency order automatically.

Step 6: Build a Pipeline

A typical multi-platform pipeline looks like:

```
CH-ENV-001 (Environment Setup)
    |
    +-- CH-BUILD-001 (Build)
    |       |
    |       +-- CH-TEST-001 (Unit Tests)
    |       |
    |       +-- CH-LINT-001 (Lint)
    |
    +-- CH-API-001 (API Health)
    |       |
    |       +-- CH-API-002 (API Flow)
    |
    +-- CH-BROWSER-001 (Browser Flow)
    |
    +-- CH-MOBILE-001 (Mobile Launch)
            |
            +-- CH-MOBILE-002 (Mobile Flow)

CH-ENV-TEARDOWN (Environment Teardown)
```

Complete Example

```go
package mychallenge

import (
    "context"
    "time"

    "digital.vasic.challenges/pkg/challenge"
    "digital.vasic.challenges/pkg/registry"
    "digital.vasic.challenges/pkg/userflow"
)

func RegisterAll(reg *registry.Registry) {
    api := userflow.NewHTTPAPIAdapter("http://localhost:8080")
    goBuild := userflow.NewGoCLIAdapter(".")

    // Environment setup
    reg.Register(userflow.NewEnvironmentSetupChallenge(
        "CH-ENV-001",
        func(ctx context.Context) error {
            // Start services, seed data, etc.
            return nil
        },
        60*time.Second,
    ))

    // Build
    reg.Register(userflow.NewBuildChallenge(
        "CH-BUILD-001",
        "Build Project",
        "Verify the project compiles",
        []challenge.ID{"CH-ENV-001"},
        goBuild,
        []userflow.BuildTarget{
            {Name: "all", Task: "./..."},
        },
    ))

    // Tests
    reg.Register(userflow.NewUnitTestChallenge(
        "CH-TEST-001",
        "Unit Tests",
        "All tests must pass",
        []challenge.ID{"CH-BUILD-001"},
        goBuild,
        []userflow.TestTarget{
            {Name: "all", Task: "./..."},
        },
    ))
```

```
    // API health
    reg.Register(userflow.NewAPIHealthChallenge(
        "CH-API-001",
        api,
        "/api/v1/health",
        200,
        []challenge.ID{"CH-ENV-001"},
    ))

    // Teardown
    reg.Register(userflow.NewEnvironmentTeardownChallenge(
        "CH-ENV-TEARDOWN",
        func(ctx context.Context) error {
            return nil
        },
    ))
}
```

Tips

- **Progress reporting**: All templates call `ReportProgress` automatically. No additional code is needed for liveness detection.
- **Assertions**: Templates produce detailed `AssertionResult` values for each step. The challenge runner aggregates these into the final report.
- **Metrics**: Templates record duration metrics in seconds. Use `ParseTestResultToMetrics` and `ParseBuildResultToMetrics` if you need to extract metrics manually.
- **Error handling**: Templates do not return Go errors from `Execute()`. Instead, failures are captured as assertion results and the challenge status is set to `StatusFailed`.